

Operator Overloading Python-ում

Ամբողջական տեղեկատու բոլոր գերբեռնելի օպերատորների և dunder method-ների մասին, օրինակներով:

1. Մաթեմատիկական գործողություններ

Թվաբանական գործողությունները հատուկ dunder method-ների օգնությամբ կարող ենք սահմանել մեր կլասների համար:

Operator	Dunder method	Նկարագրություն	Օրինակ
+	<code>__add__(self, other)</code>	գումարում	<code>Vector(1,2) + Vector(3,4)</code>
-	<code>__sub__(self, other)</code>	հանում	<code>Vector(5,6) - Vector(1,2)</code>
*	<code>__mul__(self, other)</code>	բազմապատկում	<code>Vector(2,3) * 4</code>
/	<code>__truediv__(self, other)</code>	բաժանում	<code>Vector(4,6) / 2</code>
//	<code>__floordiv__(self, other)</code>	ամբողջ բաժանում	<code>Vector(7,9) // 2</code>
%	<code>__mod__(self, other)</code>	մնացորդ	<code>Vector(7,9) % 3</code>
**	<code>__pow__(self, other)</code>	աստիճան	<code>Vector(2,3) ** 2</code>

Օրինակ — Vector կլաս

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other): # v1 + v2
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other): # v1 - v2
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar): # v * 3
        return Vector(self.x * scalar, self.y * scalar)

    def __truediv__(self, scalar): # v / 2
```

```

        return Vector(self.x / scalar, self.y / scalar)

    def __pow__(self, n):          # v ** 2
        return Vector(self.x ** n, self.y ** n)

    def __floordiv__(self, scalar): # v // 2
        return Vector(self.x // scalar, self.y // scalar)

    def __mod__(self, scalar):     # v % 3
        return Vector(self.x % scalar, self.y % scalar)

    def __str__(self):
        return f'Vector({self.x}, {self.y})'

v1 = Vector(4, 6)
v2 = Vector(1, 2)
print(v1 + v2)    # Vector(5, 8)
print(v1 - v2)    # Vector(3, 4)
print(v1 * 3)     # Vector(12, 18)
print(v1 / 2)     # Vector(2.0, 3.0)

```

2. Համեմատության օպերատորներ

Կլասի օբյեկտները կարելի է համեմատել, եթե սահմանենք համապատասխան dunder method-ները:

Operator	Dunder method	Նկարագրություն	Օրինակ
==	<code>__eq__(self, other)</code>	հավասար	<code>Temperature(20) == Temperature(20)</code>
!=	<code>__ne__(self, other)</code>	անհավասար	<code>Temperature(20) != Temperature(30)</code>
<	<code>__lt__(self, other)</code>	փոքր	<code>Temperature(10) < Temperature(20)</code>
>	<code>__gt__(self, other)</code>	մեծ	<code>Temperature(30) > Temperature(20)</code>
<=	<code>__le__(self, other)</code>	փոքր կամ հավասար	<code>Temperature(20) <= Temperature(20)</code>
>=	<code>__ge__(self, other)</code>	մեծ կամ հավասար	<code>Temperature(30) >= Temperature(20)</code>

Օրինակ — Temperature կլաս

```

class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

```

```

def __eq__(self, other):    # t1 == t2
    return self.celsius == other.celsius

def __ne__(self, other):   # t1 != t2
    return self.celsius != other.celsius

def __lt__(self, other):   # t1 < t2
    return self.celsius < other.celsius

def __gt__(self, other):   # t1 > t2
    return self.celsius > other.celsius

def __le__(self, other):   # t1 <= t2
    return self.celsius <= other.celsius

def __ge__(self, other):   # t1 >= t2
    return self.celsius >= other.celsius

t1 = Temperature(20)
t2 = Temperature(30)
print(t1 < t2)    # True
print(t1 == t2)   # False
print(t1 != t2)   # True
temps = [Temperature(30), Temperature(10), Temperature(20)]
print(sorted(temps, key=lambda t: t.celsius)) # [10, 20, 30]

```

3. In-place վերագրում (+=, -= ...)

In-place operator-ները փոփոխում են օբյեկտն ուղղակիորեն: Եթե չսահմանվեն, Python-ն օգտագործում է `__add__`-ի արդյունքը:

Operator	Dunder method	Նկարագրություն	Օրինակ
<code>+=</code>	<code>__iadd__(self, other)</code>	գումարում	<code>counter += 5</code>
<code>-=</code>	<code>__isub__(self, other)</code>	հանում	<code>counter -= 3</code>
<code>*=</code>	<code>__imul__(self, other)</code>	բազմապատկում	<code>counter *= 2</code>
<code>/=</code>	<code>__itruediv__(self, other)</code>	բաժանում	<code>counter /= 2</code>
<code>//=</code>	<code>__ifloordiv__(self, other)</code>	ամբողջ. բաժ.	<code>counter //= 3</code>

<code>%=</code>	<code>__imod__(self, other)</code>	մնացորդ	<code>counter %= 4</code>
<code>**=</code>	<code>__ipow__(self, other)</code>	աստիճան	<code>counter **= 2</code>

Օրինակ — Counter կլաս

```
class Counter:
    def __init__(self, value=0):
        self.value = value

    def __iadd__(self, amount):    # c += 5
        self.value += amount
        return self    # կարևոր է վերադարձնել self

    def __isub__(self, amount):    # c -= 3
        self.value -= amount
        return self

    def __imul__(self, factor):    # c *= 2
        self.value *= factor
        return self

    def __str__(self):
        return f'Counter({self.value})'

c = Counter(10)
c += 5
print(c)    # Counter(15)
c -= 3
print(c)    # Counter(12)
c *= 2
print(c)    # Counter(24)
```

4. Unary օպերատորներ

Unary operator-ները կիրառվում են մեկ օբյեկտի վրա (ոչ երկուսի):

Operator	Dunder method	Նկարագրություն	Օրինակ
<code>-x</code>	<code>__neg__(self)</code>	բացասական	<code>-money</code>
<code>+x</code>	<code>__pos__(self)</code>	դրական	<code>+money</code>

abs()	<code>__abs__(self)</code>	բացարձակ արժեք	<code>abs(money)</code>
~x	<code>__invert__(self)</code>	bit invert	<code>~flags</code>
round()	<code>__round__(self, n)</code>	կլորացում	<code>round(money, 2)</code>
int()	<code>__int__(self)</code>	ամբ. թիվ	<code>int(money)</code>
float()	<code>__float__(self)</code>	float	<code>float(money)</code>

Օրինակ — Money կլաս

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __neg__(self):    # -money
        return Money(-self.amount)

    def __pos__(self):    # +money
        return Money(abs(self.amount))

    def __abs__(self):    # abs(money)
        return Money(abs(self.amount))

    def __round__(self, n=0): # round(money, 2)
        return Money(round(self.amount, n))

    def __int__(self):    # int(money)
        return int(self.amount)

    def __float__(self):  # float(money)
        return float(self.amount)

    def __str__(self):
        return f'{self.amount} AMD'

m = Money(-5000)
print(abs(m))    # 5000 AMD
print(-m)        # 5000 AMD
print(int(m))    # -5000
```

5. Bit գործողություններ

Bit operator-ները հիմնականում օգտագործվում են ֆլագների (flags) կամ ցածր մակարդակի գործողությունների համար:

Operator	Dunder method	Նկարագրություն	Օրինակ
&	<code>__and__(self, other)</code>	AND	<code>perm1 & perm2</code>
	<code>__or__(self, other)</code>	OR	<code>perm1 perm2</code>
^	<code>__xor__(self, other)</code>	XOR	<code>perm1 ^ perm2</code>
<<	<code>__lshift__(self, n)</code>	ձախ shift	<code>flags << 1</code>
>>	<code>__rshift__(self, n)</code>	աջ shift	<code>flags >> 1</code>
&=	<code>__iand__(self, other)</code>	AND in-place	<code>perm &= mask</code>
=	<code>__ior__(self, other)</code>	OR in-place	<code>perm = Permission.WRITE</code>

Օրինակ — Permission (թույլտվություններ) կլաս

```
class Permission:
    READ  = 0b100  # 4
    WRITE = 0b010  # 2
    EXEC  = 0b001  # 1

    def __init__(self, flags=0):
        self.flags = flags

    def __or__(self, other):  # p1 | p2  (ավելացնել թույլտ.)
        return Permission(self.flags | other.flags)

    def __and__(self, other):  # p1 & p2  (ստուգել)
        return Permission(self.flags & other.flags)

    def __xor__(self, other):  # p1 ^ p2  (toggle)
        return Permission(self.flags ^ other.flags)

    def __lshift__(self, n):  # p << 1
        return Permission(self.flags << n)

    def __rshift__(self, n):  # p >> 1
        return Permission(self.flags >> n)

    def __str__(self):
        return bin(self.flags)
```

```
p = Permission(Permission.READ)
p2 = Permission(Permission.WRITE)
print(p | p2)    # 0b11  (READ + WRITE)
print(p & p2)    # 0b0    (ոչ մեկը չի հասկում)
```

6. Container / հաջորդականություն

Այս operator-ների օգնությամբ կարող ենք ստեղծել մեր սեփական list-ի կամ dict-ի նման կլասներ:

Operator	Dunder method	Նկարագրություն	Օրինակ
obj[i]	<code>__getitem__(self, key)</code>	կարդալ	<code>library[0]</code>
obj[i]=x	<code>__setitem__(self, key, val)</code>	գրել	<code>library[0] = 'x'</code>
del obj[i]	<code>__delitem__(self, key)</code>	ջնջել	<code>del library[0]</code>
x in obj	<code>__contains__(self, item)</code>	պարունակում է	<code>'a' in library</code>
len()	<code>__len__(self)</code>	երկարություն	<code>len(library)</code>
iter()	<code>__iter__(self)</code>	for loop	<code>for x in library</code>
next()	<code>__next__(self)</code>	հաջորդ էլեմենտ	<code>next(iterator)</code>

Օրինակ — Library կլաս

```
class Library:
    def __init__(self):
        self.books = []

    def __getitem__(self, index):
        return self.books[index]

    def __setitem__(self, index, value):
        self.books[index] = value

    def __delitem__(self, index):
        del self.books[index]

    def __contains__(self, item):
        return item in self.books

    def __len__(self):
```

```

        return len(self.books)

    def __iter__(self):
        self._index = 0
        return self

    def __next__(self):
        if self._index < len(self.books):
            result = self.books[self._index]
            self._index += 1
            return result
        raise StopIteration

```

7. Հատուկ / repr դunder method-ներ

Այս method-ները կառավարում են, թե ինչպես Python-ն «տեսնում» ու «կատարում» է օբյեկտը:

Operator	Dunder method	Նկարագրություն	Օրինակ
str()	<code>__str__(self)</code>	<code>print(obj)</code>	<code>print(my_obj)</code>
repr()	<code>__repr__(self)</code>	debug	<code>repr(my_obj)</code>
bool()	<code>__bool__(self)</code>	True/False	<code>if my_obj:</code>
hash()	<code>__hash__(self)</code>	dict key	<code>d[my_obj] = 1</code>
obj()	<code>__call__(self, ...)</code>	Ֆունկցիայի պես կանչ	<code>my_obj(5)</code>
obj.x	<code>__getattr__(self, name)</code>	attribute	<code>obj.unknown</code>
with	<code>__enter__</code> / <code>__exit__</code>	context manager	<code>with obj as x:</code>

Օրինակ — SmartList կլաս

```

class SmartList:
    def __init__(self, *args):
        self._items = list(args)

    def __str__(self):  # print(obj)
        return f'SmartList: {self._items}'

    def __repr__(self):  # debug-ի ժամանակ
        return f'SmartList({self._items})'

    def __bool__(self):  # if obj:

```



```

        return len(self._items) > 0

    def __hash__(self): # dict-ի key
        return hash(tuple(self._items))

    def __call__(self, index): # obj(0)
        return self._items[index]

    def __enter__(self): # with obj:
        print('Բացնւմ...')
        return self

    def __exit__(self, *args): # with-ի վերջ
        print('Փակնւմ...')

s1 = SmartList(1, 2, 3)
print(s1)           # SmartList: [1, 2, 3]
print(bool(s1))     # True
print(s1(1))        # 2   (call-ի արդիւթեան)
with s1:
    pass            # Բացնւմ... Փակնւմ...

```